

K-LPN 생성 ~ AGV 호출 프로세스

HD인프라코어 WMS — 키팅 완료 이후 물류 지시 흐름

작성일 2026-09-07 대상 P26_HD_INFRACORE_WMS 범위 3중 검증 → K-LPN 발행 → 스테이션 배정 → RFID → AGV 호출

구현 상태를 구현 일부 미구현 으로 표시한다.

1. 전체 흐름

0

키팅 지시 발행

구현

POST /issue/kit — 피킹·세척·적치 완료 확인 후 kit_table WAIT → ISSUED

▼ 작업자가 태블릿에서 K-LPN 발행 버튼 클릭

1

3중 검증

구현

POST /verify/kit — W/D-LPN 스캔, 서열·호기 / 공정 / 재고상태

▼ 필요한 용기가 전부 PASS

2

K-LPN 생성

구현

— 트랜잭션 1

채번 → lpn_master INSERT(KIT_COMP) → W/D 자재 통합 → 원본 소멸 → kit_table KITTED

▼ 커밋 완료, 이후 실패해도 키팅은 되돌아가지 않는다

3

스테이션 배정

일부

— HTTP + 트랜잭션 2

ACS REST 로 station_no 수신 → kit_table 저장 → K-LPN KIT_COMP → ISSUED

▼ ISSUED 가 되어야 RFID 목록에 노출된다

4

RFID 태그 쓰기

일부

웹소켓으로 목록 push → 핸디가 태그 write → 완료 회신 → rfid_yn = 1

▼ 태그까지 완료된 건만 AGV 호출 대상

5

AGV 호출

미구현

— 소켓, 1초 주기

ACS 프로세스가 대상 건에 agv_cmd 전송, 응답의 상태값으로 K-LPN 상태 변경

▼ 상태가 바뀌면 대상에서 빠진다 = 호출 완료

6

AGV 이송 / 도착

미구현

AGV 가 목적지(투입 포트)로 이송, 도착 판정 경로 미확정

2. 단계별 상세

단계 0 — 키팅 지시 발행

항목	내용
진입점	<code>POST /issue/kit</code> → <code>kit_service.issue_kit()</code>
발행 조건	<code>_check_ready()</code> — 아래를 모두 만족해야 한다
상태 전이	<code>kit_table.STATUS</code> : <code>WAIT</code> → <code>ISSUED</code>

- 미완료 피킹 0건 (`pick_table.STATUS = 'PICKED'`)
- `W_LPN_SEQ` 가 있으면 → `process_status = 'WASH_COMP'` (세척 완료)
- `D_LPN_SEQ` 가 있으면 → `location_seq` 존재 (버퍼 적치 완료)
- `HOLD_YN = 0`, `LIFECYCLE_STATUS = 'ACTIVE'`

설계 의도 — 발행 조건은 "W/D 둘 다 있냐"가 아니라 "필요한 게 다 됐냐"이다. `W_LPN_SEQ` 가 NULL 이면 세척 자재가 없는 공정이므로 세척 완료를 기다리지 않는다.

단계 1 — 3중 검증

진입점 `POST /verify/kit` → `kit_service.verify_kit()`. W/D-LPN 을 스캔할 때마다 1회씩 호출된다.

구분	검증 항목	통과 조건
①	서열 / 호기	스캔 LPN 의 <code>kit_seq</code> , <code>engine_no</code> , <code>engine_seq_no</code> 가 지시와 일치
②	공정 (OP)	LPN 의 <code>proc_code</code> = 지시 공정
③	재고 상태	W : <code>WASH_COMP</code> / D : <code>PICK_COMP</code> + 로케이션 존재 공통 : <code>lifecycle_status = 'ACTIVE'</code>

- PASS / FAIL 무관하게 **전 건**을 `kit_verify_log` 에 기록한다. 오조립 클레임 시 투입 이력 증빙이 필요하기 때문.
- 첫 PASS 가 곧 키팅 착수 — `kit_table ISSUED` → `KITTING`
- 완료 판정 : `need` = (`W_LPN_SEQ`, `D_LPN_SEQ`) 중 NOT NULL 개수, `done` = PASS 한 LPN 의 DISTINCT 개수.
`done >= need` 이면 단계 2 로 진행

단계 2 — K-LPN 생성 (트랜잭션 1)

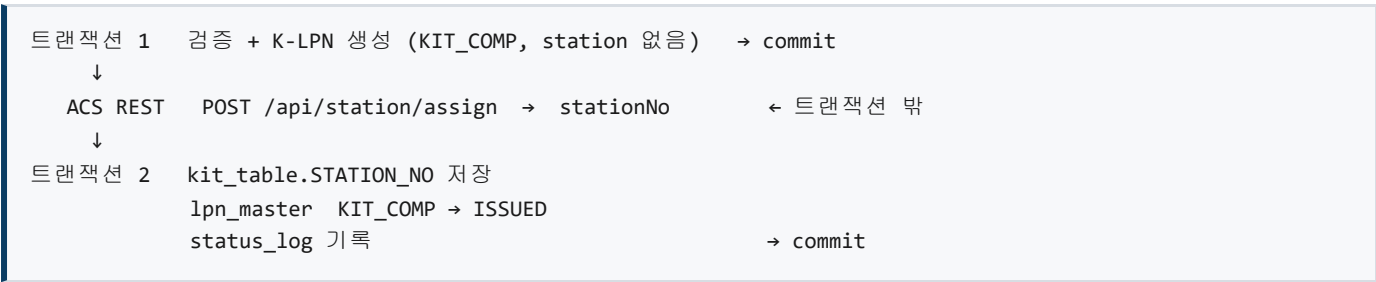
`_create_k_lpn()`. 검증과 **같은 트랜잭션** 안에서 실행된다.

순서	처리	내용
1	채번	<code>make_lpn_code(cur, 'K')</code> — <code>lpn_seq</code> 테이블 UPDLOCK. 형식 <code>K + yymmdd + 5자리</code> (예 <code>K25090700001</code>), 일 99,999 한도
2	<code>lpn_master</code> INSERT	<code>lpn_type='K'</code> , <code>process_status='KIT_COMP'</code> , <code>rfid_yn</code> 미지정(NULL), <code>kit/order/engine/proc/engine_seq</code> 스냅샷. station_no 없음
3	<code>lpn_detail</code> 복사	W + D 의 자재를 <code>item_seq</code> 기준 SUM 하여 K-LPN 하나로 통합
4	원본 소멸	<code>lpn_txn</code> 에 KT 이력 기록 → <code>current_qty = 0</code> → <code>process_status='CONSUMED'</code> , <code>lifecycle_status='INACTIVE'</code>
5	키팅 완료	<code>kit_table.K_LPN_SEQ</code> 설정 + <code>STATUS='KITTED'</code> + <code>status_log</code>

원본 수량을 비우지 않으면 재고가 이중 계상된다. 4번은 생략할 수 없다. 이후 AMR 은 K-LPN 만 추적한다.

단계 3 — 스테이션 배정 (HTTP + 트랜잭션 2)

트랜잭션 1 이 커밋된 뒤, `kit_service.assign_station()` 이 트랜잭션 밖에서 실행된다.



왜 ACS 호출을 트랜잭션 안에 넣지 않는가

- ① 응답이 timeout(5초)까지 걸리는 동안 DB 락을 잡게 된다.
- ② ACS 무응답으로 롤백되면 **3중 검증 로그까지 같이 날아간다**. "FAIL 도 반드시 남긴다"는 원칙이 깨진다.

왜 INSERT 를 먼저 하는가 (station_no NULL 허용)

순서를 뒤집어 ACS 호출 → INSERT 로 하면, 그 사이에 장애가 났을 때 **ACS 는 배차했는데 WMS 에는 기록이 없는 상태**가 된다. 스테이션이 유령 점유되고 조회할 방법조차 없다.

외부 시스템 호출과 로컬 커밋은 원자적으로 묶을 수 없으므로, **우리 기록을 항상 먼저 남긴다**.

실패 시 복구

상황	처리
ACS 무응답 / 오류	<code>assign_station()</code> 이 예외를 올리지 않고 None 반환. API 응답은 <code>station_no: null</code> + "키팅 완료, 배차 대기". 키팅은 이미 끝났는데 예외를 터뜨리면 작업자에게 키팅 실패로 보인다.
남은 상태	K-LPN 은 <code>KIT_COMP</code> + <code>STATION_NO IS NULL</code> 로 남는다. RFID 목록에도 AGV 대상에도 들어가지 않는다.
자동 복구	스케줄러 <code>station_assign</code> 잡이 10초 주기 로 <code>SELECT_KLPN_NO_STATION</code> 대상을 재요청한다.

단계 4 — RFID 태그 쓰기

항목	내용
목록 전달	웹소켓 <code>/ws/rfid</code> . <code>broadcast_loop</code> 가 2초 주기로 조회 후 접속 중인 핸디 전체에 push. 스냅샷이 바뀌었을 때만 전송.
대상 조건	<code>lpn_type='K' AND process_status='ISSUED' AND lifecycle_status='ACTIVE'</code> <code>AND rfid_yn = 0</code>
완료 회신	현재 : 웹소켓 <code>RFID_DONE</code> → <code>rfid_yn=1</code> 변경 예정 : HTTP API 로 전환 (웹소켓은 목록 push 전용 단방향)
중복 방지	핸디 2대에 같은 목록이 나간다. <code>MARK_RFID_DONE</code> 의 <code>rfid_yn = 0</code> 조건으로 걸러지고, 두 번째 회신은 <code>rowcount=0</code> 으로 무시.

rfid_yn 은 재시도 대상이 아니다. 태그 쓰기는 사람이 하는 작업이라 WMS 가 재전송할 것이 없다. 실패하면 작업자가 다시 쓴다. WMS 는 "할 일 목록"만 보여주면 된다.

단계 5 — AGV 호출 (미구현)

ACS 소켓 통신으로 나간다. 재시도 루프는 `acs_handler` (ACS 프로세스) 안에 둔다.

항목	내용
실행 위치	ACS 프로세스 내 데몬 스레드. 이미 <code>send_queue</code> 를 갖고 있고 <code>init_db</code> 도 하며, 응답 핸들러가 같은 프로세스에 있다. → 웹 프로세스에 큐를 넘길 필요가 없어진다.
주기	1초
대상 조건	<code>process_status='ISSUED' AND STATION_NO IS NOT NULL AND rfid_yn = 1</code>
종료 조건	응답의 상태값으로 K-LPN <code>process_status</code> 를 변경 → 다음 주기 대상에서 자동으로 빠짐 = 호출 완료
재시도	별도 로직 불필요. 응답이 유실되면 상태가 안 바뀌므로 다음 주기에 또 나간다. 1초 주기 자체가 재시도 이다.

지시 모델이 아니라 상태 모델이다.

"1회 전송 후 ack 대기, 실패 시 재전송"(지시 모델)이 아니라, "조건이 유지되는 동안 계속 전송, 상태가 바뀌면 중단"(상태 모델)로 간다. 전문 유실에 구조적으로 강하고 `agv_call_yn` 같은 별도 플래그가 필요 없다.

단, **ACS 가 같은 lpn_code 에 대해 역등해야 한다** — 반복 요청에 중복 배차하면 안 된다.

단계 6 — 이송 / 도착

AGV 가 `destination` (투입 포트)으로 K-LPN 을 이송한다. 도착을 WMS 가 어떻게 인지하는지는 **미확정**이다. 후보:

- ACS 가 `rfid_read` 전문 송신 (도착지 리더가 K-LPN 을 읽음) — `_handle_rfid_read` 가 TODO 로 남아 있어 이 경로로 보인다
- ACS 가 별도 도착 전문 송신
- `agv_cmd` 응답의 `status` 값이 `ARRIVED` 로 바뀜

3. 상태 전이 요약

대상	전이	시점
kit_table.STATUS	WAIT → ISSUED	키팅 지시 발행
	ISSUED → KITTING	3중 검증 첫 PASS
	KITTING → KITTED	K-LPN 생성
lpn_master (W/D)	WASH_COMP / PICK_COMP → CONSUMED lifecycle ACTIVE → INACTIVE	K-LPN 생성 시 소멸
lpn_master (K)	(신규) → KIT_COMP	K-LPN 생성. 배차 대기
	KIT_COMP → ISSUED	스테이션 배정 완료
	ISSUED → (ACS 상태 매핑값)	AGV 호출 응답 수신 미확정
lpn_master.rfid_yn	0/NULL → 1	핸디 태그 쓰기 완료 회신
kit_table.STATION_NO	NULL → 값	ACS 스테이션 배정

4. 전문 규격 (합의 진행 중)

봉투에 type 을 추가해 요청 / 응답을 구분한다. 현재는 응답이 전부 cmd:"ack" + data.of 로 뭉쳐 들어와 두 군데를 봐야 어느 전문의 응답인지 알 수 있다.

AGV 호출 요청 (WMS → ACS)

```
{
  "type": "request",
  "cmd": "agv_cmd",
  "data": {
    "lpn_code": "K25090700001",
    "station_no": "ST-02",
    "destination": "AH-0010"
  },
  "time": "2026-09-07 14:03:12"
}
```

응답 — 성공 (ACS → WMS)

```
{
  "type": "response",
  "cmd": "agv_cmd",
  "data": {
    "result": true,
    "lpn_code": "K25090700001",
    "status": "MOVING"
  },
  "time": "2026-09-07 14:03:12"
}
```

응답 — 실패

```
{ "type": "response", "cmd": "agv_cmd",  
  "data": { "result": false,  
            "lpn_code": "K25090700001",  
            "reason": "스테이션 점유 중"},  
  "time": "2026-09-07 14:03:12" }
```

규칙

- `result` 는 **응답 전용**. 요청에는 넣지 않는다.
- 응답에는 성공/실패 무관하게 `lpn_code` 가 있어야 한다. 없으면 어느 K-LPN 의 상태를 바꿀지 알 수 없다.
- `type == "response"` 인 전문에는 절대 응답하지 않는다 (무한 핑퐁 방지).
- `req_id` 는 두지 않는다. 응답 처리가 역등("이 `lpn_code` 를 이 상태로")이라 몇 번째 요청의 응답인지가 의미를 갖지 않는다.

배포 주의 — 봉투 규격 변경은 WMS / ACS **양쪽 동시 배포**가 필요하다. 한쪽만 바꾸면 전 전문이 미등록으로 떨어진다. 과도기가 필요하면 `type` 없는 전문을 `request` 로 간주하는 하위 호환 처리를 `dispatch` 에 한시적으로 둔다.

5. 예외 / 복구 시나리오

상황	결과	복구
3중 검증 FAIL	로그만 남고 상태는 안 바뀜	작업자가 올바른 용기를 다시 스캔
ACS 무응답 (스테이션 배정)	K-LPN 은 KIT_COMP + station NULL 로 남음. 키팅 자체는 완료	스케줄러 10초 주기 재요청 구현
ACS 소켓 끊김 (AGV 호출)	전문이 조용히 버려짐. 재전송 없음	1초 주기가 곧 재시도. 재연결되면 자동 복구
AGV 응답 유실	K-LPN 상태가 안 바뀜	다음 주기에 재전송. ACS 먹등성 전제
핸디 2대 중복 write	두 번째 회신 rowcount = 0	무시. 오류 아님
ACS 가 정의되지 않은 status 값 반환	매핑에 없으면 거부 + 경고	그대로 DB 에 넣으면 K-LPN 이 어떤 쿼리에도 안 잡 히는 상태로 빠진다

6. 미확정 항목

#	항목	내용
1	ACS 스테이션 API 스펙	경로 / 요청 필드 / 응답 필드. 현재 POST /api/station/assign → stationNo 로 가정하고 TODO 표기
2	AGV status 값 집합	MOVING 외 전체 목록. WMS process_status 매핑 필요
3	AGV 도착 인지 경로	rfid_read 인지, 별도 전문인지, status 변화인지
4	station_no 타입	숫자(2) 인지 문자열("ST-02") 인지. DB 컬럼과 통일 필요
5	destination 접두어	proc_param.py 매핑은 AH- 로 시작. AF- 표기가 나온 적이 있어 확인 필 요
6	ACS 먹등성	같은 lpn_code 반복 요청 시 중복 배치하지 않는가. 상태 모델의 전제 조건
7	RFID 완료 회신 방식	웹소켓 RFID_DONE 유지 vs HTTP API 전환

7. 관련 소스

파일	역할
<code>src/routers/kit_router.py</code>	키팅 발행 / 3중 검증 API
<code>src/services/kit_service.py</code>	발행 조건 판정, 3중 검증, K-LPN 생성, 스테이션 배정
<code>src/queries/kit_query.py</code>	위 전 과정의 SQL
<code>src/services/acs_service.py</code>	ACS REST 클라이언트 — 조회성 요청 (스테이션 배정)
<code>src/acs_handler.py</code>	ACS 소켓 창구 — 지시성 전문 (AGV 호출, RFID)
<code>src/module/socket_module/message_server.py</code>	봉투 규격 / 프레임 / 큐 (공용)
<code>src/services/ws/ws_service.py</code>	핸디 RFID 웹소켓 — 목록 push
<code>src/scheduler/jobs.py</code>	배차 대기 K-LPN 재요청 잡
<code>src/param/proc_param.py</code>	공정 → 투입 포트 목적지 매핑